

Kibo-RPC 2025 分享

林穎沛

Overview

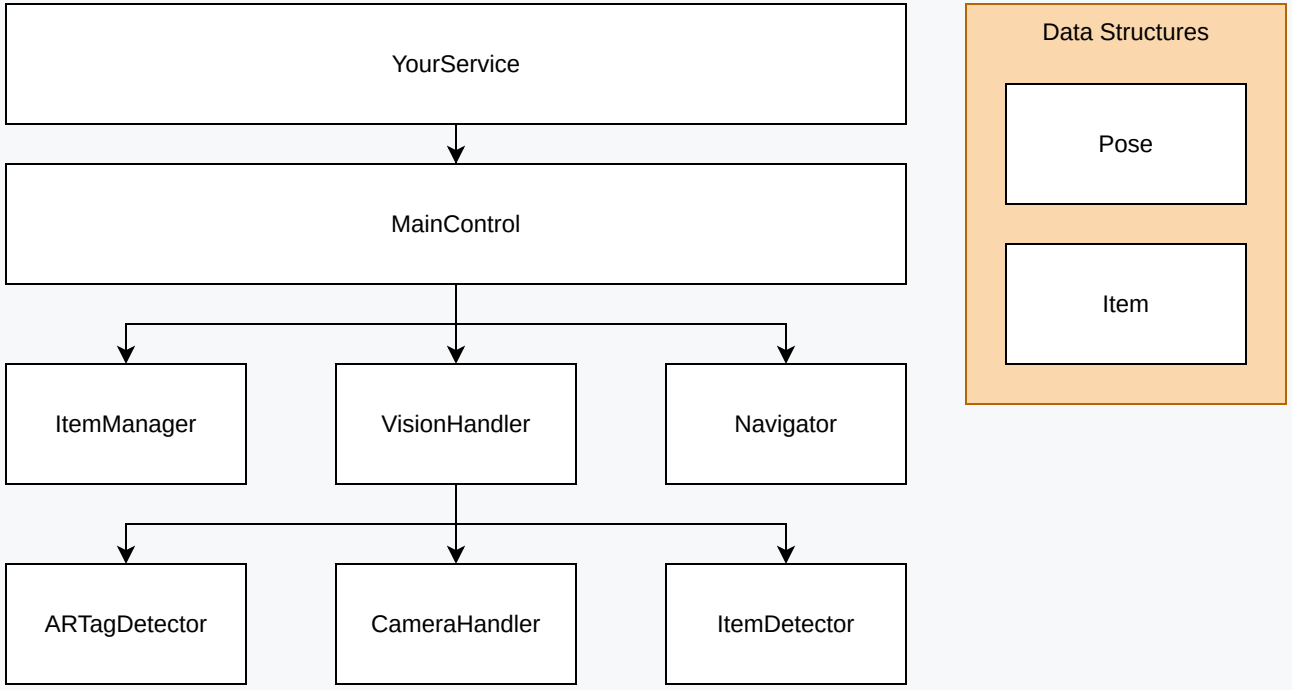
1. **系統**｜任務如何拆成流程與模組
2. **計算**｜座標、位姿與安全目標
3. **策略**｜路線取捨與失敗處理
4. **工具**｜開發、測試與版本驗證

流程

```
任務目標
  ↓
規劃下一個目標位姿
  ↓
Astrobee 執行 moveTo
  ↓
讀取目前位置、姿態與任務回傳
  ↓
檢查執行結果
  ├─ 正常 → 下一狀態
  └─ 異常 → 重試 / 恢復
```

程式依照「觀測 → 決策 → 動作 → 驗證」的循環推進

模組


YourService
MainControl
ItemManager
VisionHandler
Navigator
ARTagDetector
CameraHandler
ItemDetector
Data Structures
Pose
Item

視覺模組

NavCam
→ 相機內參去畸變
→ ArUco 定位 + 透視校正後裁切目標區域
→ 影像縮放與前處理
→ TFLite 物件辨識
→ 合併、過濾候選結果
→ Item + Pose

位置轉換

已知 B 在 A 中的位置與旋轉，以及 C 在 B 中的位置：

$$p_C^A = p_B^A + R_B^A p_C^B$$

C 在 B 座標中的位置
→ 依 B 相對 A 的姿態旋轉
→ 加上 B 在 A 中的位置
→ 得到 C 在 A 座標中的位置

轉換矩陣

- 把「旋轉後再平移」合併成一次矩陣乘法：

$$\begin{bmatrix} x^A \\ y^A \\ z^A \\ 1 \end{bmatrix} = \underbrace{\left[\begin{array}{ccc|c} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ \hline 0 & 0 & 0 & 1 \end{array}\right]}_{T_B^A} \begin{bmatrix} x^B \\ y^B \\ z^B \\ 1 \end{bmatrix}$$

- 旋轉 r：改變座標軸方向
- 平移 t：B 在 A 中的位置

座標系轉換

$$\text{OpenCV Camera} \xrightarrow{T_C^N} \text{NavCam} \xrightarrow{T_N^B} \text{Astrobee Body} \xrightarrow{T_B^W} \text{World}$$

$$\begin{bmatrix} p_{Tag}^W \\ 1 \end{bmatrix} = T_B^W \; T_N^B \; T_C^N \begin{bmatrix} p_{Tag}^C \\ 1 \end{bmatrix}$$

路線取捨

已知時間分數每秒約 **0.2 分**，繞行 Oasis 的淨收益：

$$\Delta S_{net} = \Delta S_{oasis} - 0.2\,\Delta t$$

因此繞路值得的條件：

$$\Delta S_{oasis} > 0.2\,\Delta t$$

例：多花 10 秒損失 2 分；若 Oasis 增益超過 5 分，仍有正收益。

策略轉折： 前期先縮短任務時間；時間已難再壓縮後，才用少量繞行換取較高總分。

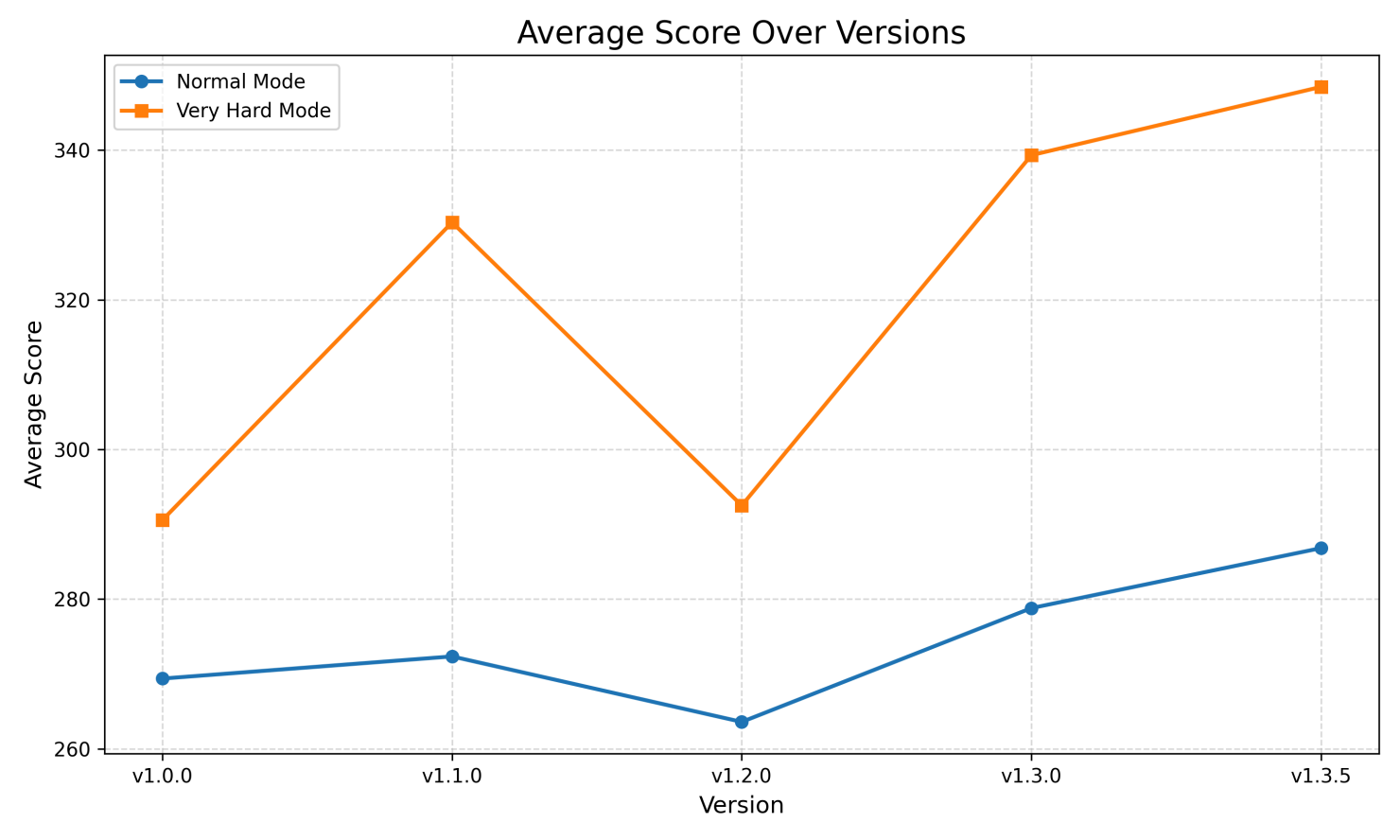
Fallbacks

風險	程式中的處理
移動指令失敗	記錄原因並有限次重試
到點仍漂移	距離、角度門檻觸發重新定位
移動後系統未穩	依目標位置設定等待時間
模組沒有有效結果	設定重試上限與後續處理
優化策略沒有成功	回到較保守的替代流程

工具

階段	工具	用途
幾何驗證	Python、NumPy、SciPy、Matplotlib	檢查拍攝點、四元數與 3D 區域
模型訓練與部署	OpenCV、Ultralytics、GPU、TFLite	合成資料、訓練、轉換與離線評估
飛行端	Java、Kibo API、OpenCV、TFLite	任務整合與實際執行
開發環境	Docker、VS Code、Git/GitHub	可重現建置、協作、版本追蹤
回歸測試	Simulator、Selenium、Excel	自動上傳、收集分數與失敗樣本

驗證



每版 30–60 次｜最終平均 286.8 分

經驗

問題	處理方式／改進方向
手冊沒有交代競賽框架細節	反編譯、讀 API 行為
模擬失敗不一定有完整 log	自己記錄階段時間、回傳值與失敗分類
組員的開發環境不同	使用 Docker 固定建置環境
重複測試耗時	自動上傳並整理測試結果

開發順序

1. 把規則整理成 **狀態、輸入、輸出、失敗條件**
2. 定義少量共用資料結構與模組介面
3. 用 Python 驗證座標、姿態、成本函數
4. 可完成整個任務的基本版本
5. 再加入捷徑、加分路線與進階策略
6. 自動化跑模擬；以平均、變異與失敗率選版本

優化順序：正確性 → 穩定性 → 速度 → 額外分數

Key Takeaways

1. 追求穩定而不是單次高分
2. 將失敗處理納入系統設計
3. 用實驗數據驗證每次優化
4. 局部最佳不代表整體最佳



關於 iTron

不只學習理論，更要親手打造機器人！

- **團隊理念**：透過實作、交流與競賽，持續挑戰自我
- **機構組**：3D 建模、機構設計、加工與組裝
- **軟體組**：程式設計、電腦視覺、路徑規劃、機器學習
- **學習方式**：新生入門教學 → 分組訓練 → 專案實戰
- **參與競賽**：TDK、ASME、Kibo-RPC


iTRon
robotic team

Q & A